Lecture 3

Entity Relationship Modeling

CSC1022 Introduction to Databases

CSC1005 Data Science & Databases
2025/2026 Semester 2

Prof. Mark Roantree
Uttaran Bera
School of Computing
@ Dublin City University

Agenda

1. **Entity Types**
2. **Introduction**
3. **Entities**
   - Entities and Their Attributes
   - Entity Types, Entity Sets, Keys, and Value Sets
   - Initial Conceptual Design of the COMPANY Database
4. **Relationships**
   - Relationship Types, Relationship Sets, Roles, and Structural Constraints
   - Relationship Degree, Role Names, and Recursive Relationships
   - Constraints on Relationship Types
   - Attributes of Relationship Types
5. **Weak Entity Types**
6. **Summary**

Learning Objectives

- How can I describe *things* using a high-level **conceptual data model**?
- How do I connect these *things*?
- Can I present my design so there is no ambiguity?
- Is it clear (there is only 1 interpretation) to others?

Conceptual Modelling

- In this section, we create a **conceptual schema** for the database, using a high-level conceptual data model.
- The conceptual schema includes detailed descriptions of the **entity types**, **relationships**, and **constraints**.
- They are expressed using the concepts provided by the high-level data model.

COMPANY Case Study

Departments

- The company is organized into **departments**.
- Each department has a unique name, a unique number, and a particular employee who manages the department.
- We keep track of the start date when that employee began managing the department.
- A department may have several locations.

Projects

- A department controls a number of **projects**, each of which has a unique name & number, and a single location.

Employees

- We store each employee's name, Social Security number, address, salary, sex, and birth date.
- An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department.
- Track the current number of hours that an employee works on each project.
- Track the direct supervisor of each employee (an employee).

Dependents

- We want to keep track of the dependents of each employee for insurance purposes.
- We keep each dependent's first name, sex, birth date, and relationship to the employee.

Figure 1 shows how the schema for this database application can be displayed by means of the graphical notation known as ER diagrams.

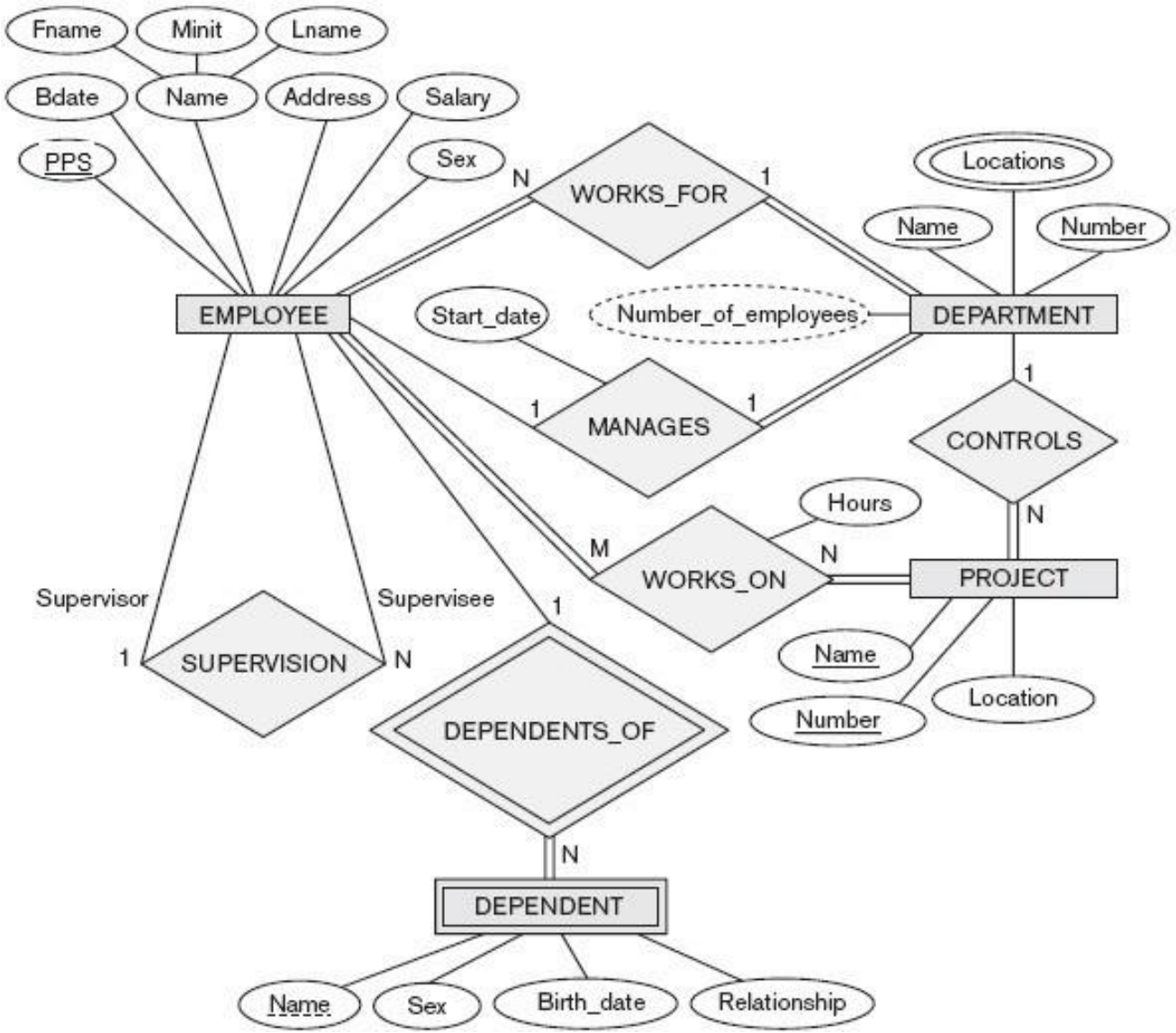


Figure 1: An ER schema diagram for the COMPANY database

Entities and Attributes

- The basic object that the ER model represents is an entity, which is a thing in the real world with an independent existence.
- An entity may be an object with a physical existence (for example, a particular person, car, house, or employee),
- or it may be an object with a conceptual existence (for instance, a company, a job, or a university course).

Attributes

- Each entity has attributes: the particular properties that describe it.
- For example, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job.
- A particular entity will have a value for each of its attributes.
- The attribute values that describe each entity become a major part of the data stored in the database.

Sample Entities

- The EMPLOYEE entity has 6 attributes: `PPS`, `Bdate`, `Name`, `Address`, `Salary` and and `Sex`
- Values could be: '3811255R', '23/02/75', 'John Kennedy', '20 Glasnevin Ave, Dublin 9', 60000,'M' respectively.
- The DEPARTMENT entity has 3 attributes: `Name`, `Location`, and `Number`.
- Values could be: 'Accounts', 'Finglas', and 'B12', respectively.

Composite versus Simple (Atomic) Attributes

- **Composite attributes** can be divided into smaller subparts, comprising attributes with independent meanings.
- For example, the `Name` attribute of the EMPLOYEE entity is subdivided into `Fname`, `City`, `Minit`, and `Lastname`.
- Attributes that are not divisible are called simple or *atomic* attributes.
- Composite attributes can form a hierarchy: `Street_address` can be further subdivided into three simple component attributes: `Number`, `Street`, and `Apartment_number`.

Composite attributes can model situations where we refer to the composite attribute as a unit but separately, refer to specific components.

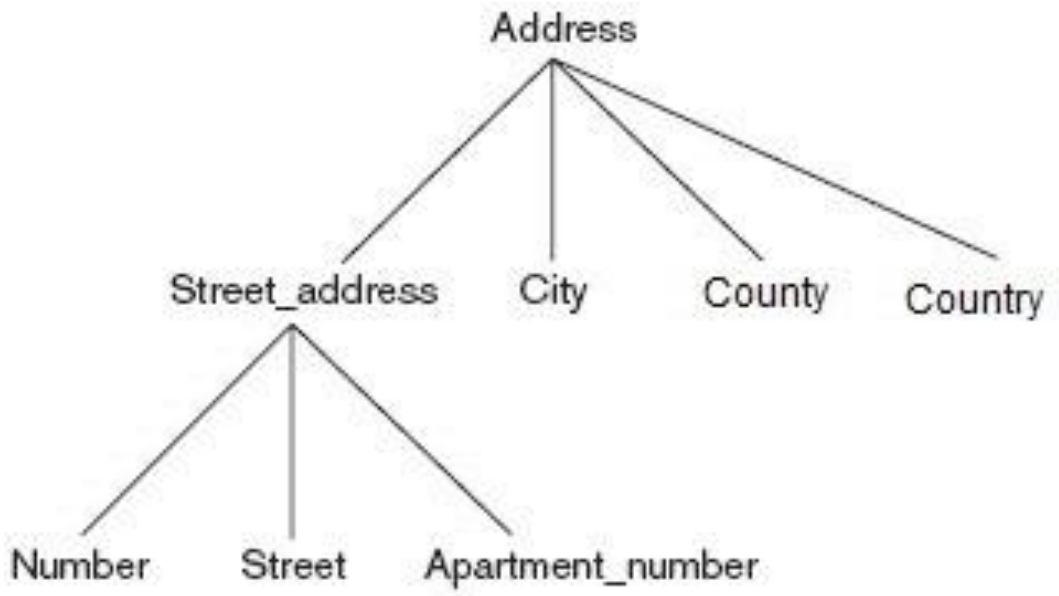


Figure 2: A hierarchy of Composite Attributes

Single-Valued versus Multivalued Attributes

- Most attributes have a single value for a particular entity; such attributes are called single-valued.
- For example, `Age` is a single-valued attribute of a person.
- In some cases an attribute can have a set of values for the same entity: for instance, a `Colors` attribute for a car, or a `College_degrees` attribute for a person.
- A person may not have a college degree, another person may have one, and a third person may have two or more degrees.

Single-Valued versus Multivalued Attributes

- Thus, different people can have different numbers of values for the `College_degrees` attribute.
- Such attributes are called multivalued.
- A multivalued attribute may have lower and upper bounds to *constrain* the number of values allowed for each individual entity.
- For example, the `Colors` attribute of a car may be restricted to have between one and three values, if we assume that a car can have three colors at most.

Stored versus Derived Attributes

- Sometimes, two (or more) attribute values are related: the `Age` and `Birth_date` attributes of a person.
- For a particular person entity, the value of `Age` can be determined from today's date and the value of that person's Birth_date.
- The `Age` attribute is a *derived* attribute, derivable from the `Birth_date` attribute, which is a *stored* attribute.
- Some attribute values can be derived from related entities. An attribute `Number_of_employees` of a DEPARTMENT entity can be derived by counting the number of employees related to (working for) that department.

NULL Values

- In some cases, a particular entity may not have an applicable value for an attribute.
- For example, a `College_degrees` attribute applies only to people with college degrees.
- For such situations, a special value called NULL is created.
- A person with no college degree would have NULL for `College_degrees`.
- NULL can also be used if we do not know the value of an attribute for a particular entity

Entity Types and Entity Sets

- A database contains groups of entities that are similar.
- For example, employee entities share the same attributes, but each entity has its own value(s) for each attribute.
- An **entity type** defines a collection (or set) of entities that have the same attributes.
- Each entity type in the database is described by its name and attributes.

Entity Sets

- The collection of all entities of a particular entity type in the database at any point in time is called an **entity set**.
- EMPLOYEE refers to both a *type* of entity and the *entire set* of employee entities in the database.

Entity types

- An **entity type** is represented as a **rectangular box** enclosing the entity type *name*.
- **Attribute names** are enclosed in **ovals** and attached to their entity type **by straight lines**.
- **Composite attributes** are attached to their *component* attributes by **straight lines**.
- **Multivalued** attributes are displayed in **double ovals**.

Sample CAR entity type using ER notation.

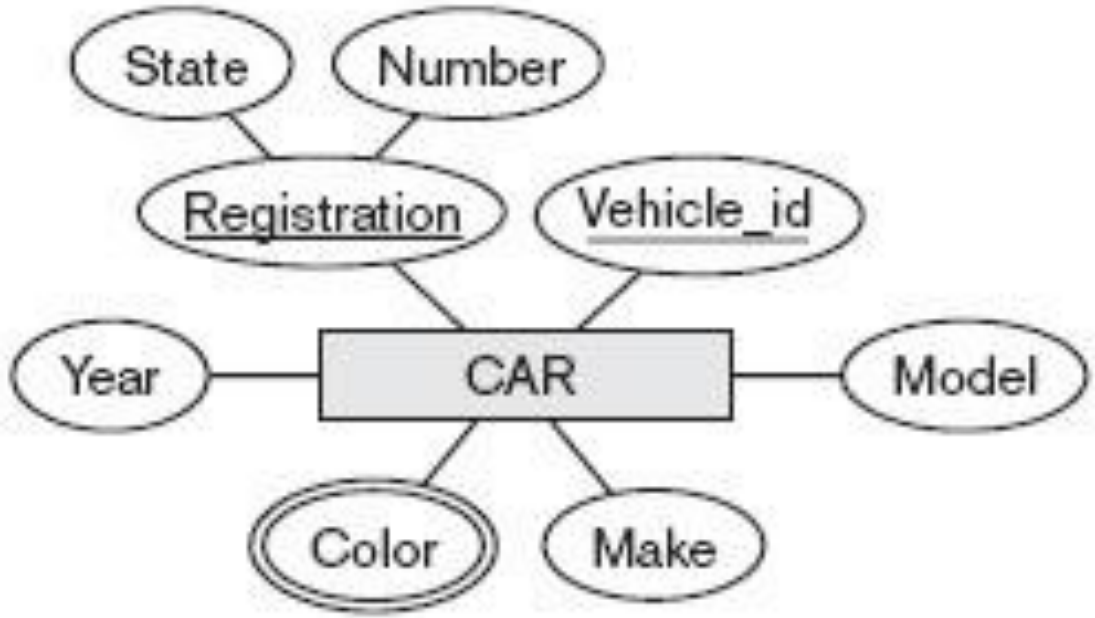


Figure 3: CAR Entity Type

Data (extension) and Data Structure (intension)

- An entity type describes the *schema* or *intension* for a set of entities that share the same structure.
- The collection of entities of a particular entity type is grouped into an entity set, which is also called the *extension* of the entity type.

Key Attributes of an Entity Type

- An important constraint on entities is the key or *uniqueness constraint* on attributes.
- An entity type usually has one or more attributes whose values are distinct for each individual entity.
- Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely.
- The Name attribute is a *key* of the COMPANY entity type because no 2 companies can have the same name.

Key Attributes

- For EMPLOYEE, the key attribute is `PPS`.
- Sometimes several attributes form the key: the combination of attribute values must be distinct.
- Here, use the ER model to define a composite attribute and designate it as a key attribute of the entity type.
- Note that such a composite key must be **minimal**: *all* component attributes must be included in the composite attribute to have the uniqueness property.
- Superfluous attributes must not be included in a key.

Key Constraints

- In ER notation, a key attribute has its name underlined inside the oval.
- This means that the preceding uniqueness property must hold for every entity set of the entity type.
- Some entity types have more than one key attribute.
- There is *no* concept of **primary key** in the ER model: it's a relational model concept!

Entity Types for COMPANY

1. DEPARTMENT with attributes `Name`, `Number`, `Locations`, `Manager`, and `Manager_start_date`. `Locations` is the only multivalued attribute.
   We can specify that both `Name` and `Number` are (separate) key attributes because each was specified to be unique.
2. PROJECT. Attributes `Name`, `Number`, `Location`, and `Controlling_department`.
   Both `Name` and `Number` are (separate) key attributes.

Preliminary Design: DEPARTMENT

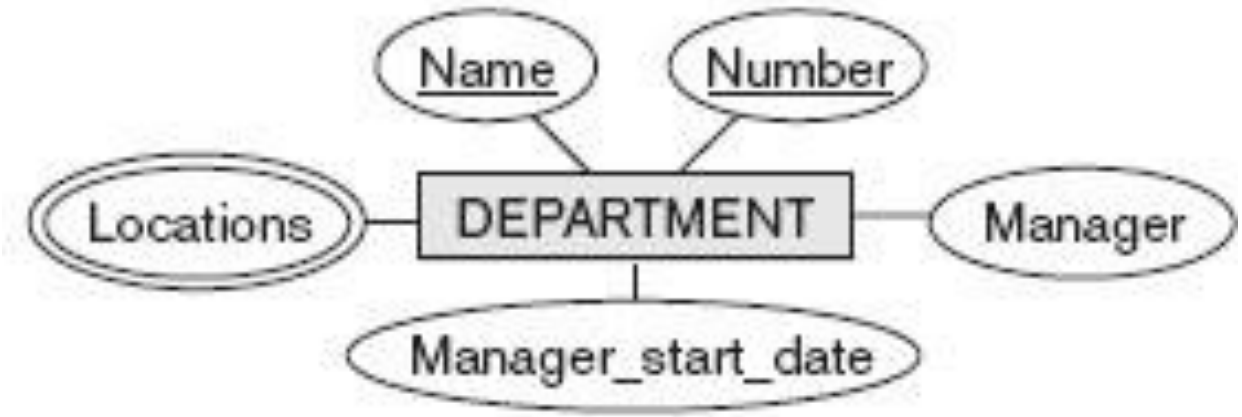


Figure 4: DEPARTMENT Entity Type

Preliminary Design: PROJECT

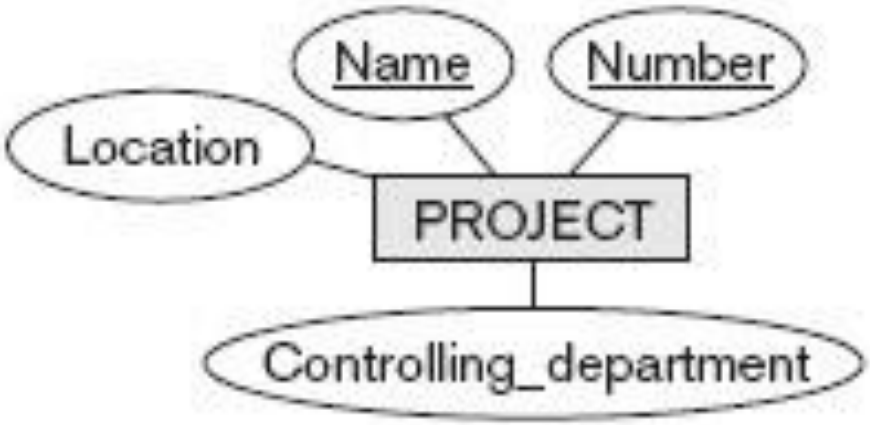


Figure 5: PROJECT Entity Type

Entity Types for COMPANY

3. EMPLOYEE. Attributes `Name`, `Ssn`, `Sex`, `Address`, `Salary`, `Birth_date`, `Department`, and `Supervisor`. Both `Name` and `Address` may be composite attributes.
4. DEPENDENT. Attributes `Employee`, `Dependent_name`, `Sex`, `Birth_date`, and `Relationship` (to the employee).

Preliminary Design: EMPLOYEE

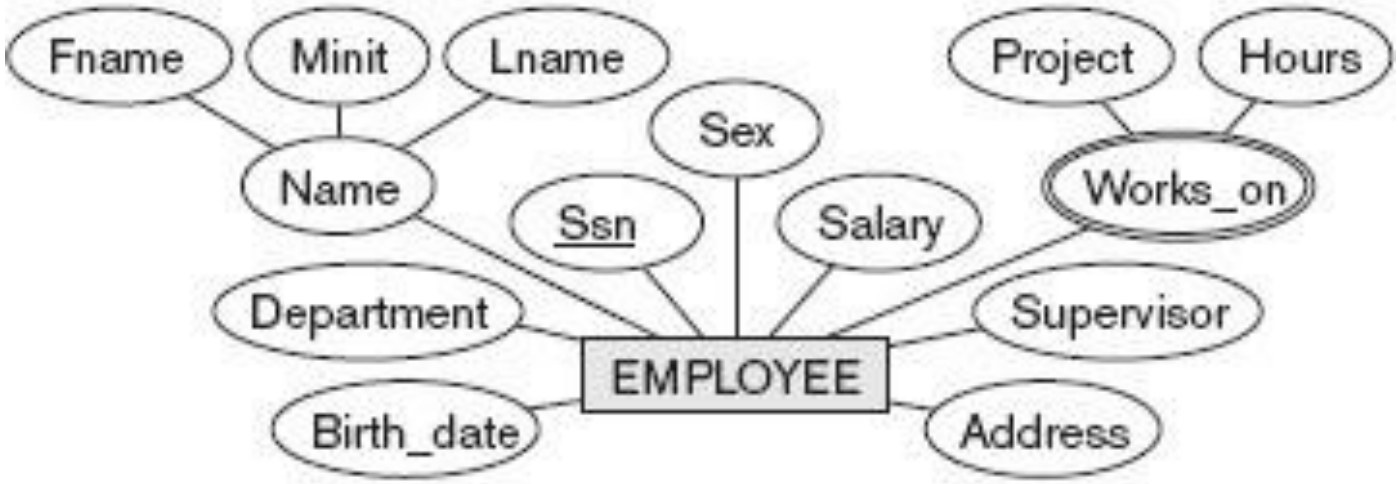


Figure 6: EMPLOYEE Entity Type

How do you recognise a complex attribute? What does the double oval represent?

Preliminary Design: DEPENDENT

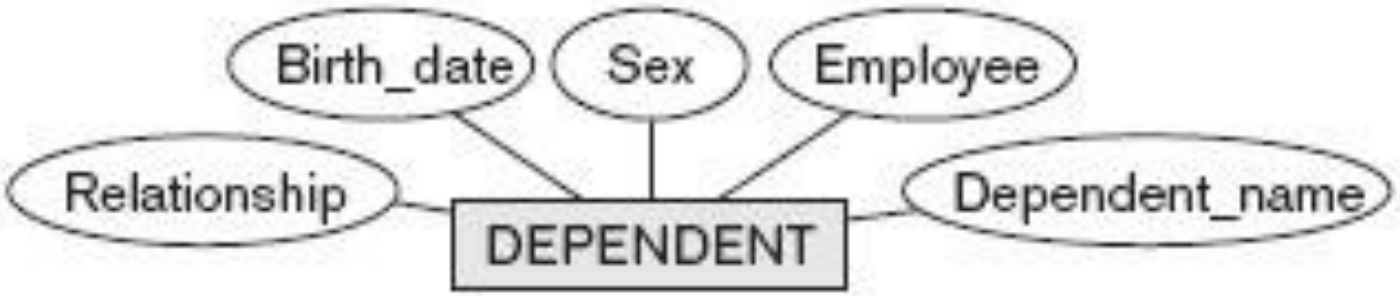


Figure 7: DEPENDENT Entity Type

- An employee can work on *several projects*: this is represented by a *multivalued composite attribute* of EMPLOYEE called `Works_on` with the simple components (`Project, Hours`).
- Alternatively, it could be represented as a multivalued composite attribute of PROJECT called `Workers` with the simple components (`Employee, Hours`).
- We choose the first alternative in Figures 4, 5, 6 and 7, which shows each of the entity types just described.

Relationship Types

- There are several implicit relationships among the various entity types.
- In fact, whenever an attribute of one entity type refers to another entity type, some relationship exists:
  - attribute Manager of DEPARTMENT refers to an employee who manages the department;
  - attribute `Controlling_department` of PROJECT refers to the department that controls the project;
  - the attribute Supervisor of EMPLOYEE refers to another employee (the one who supervises this employee); and so on.
- In the ER model, these references should not be represented as attributes but as *relationships*.

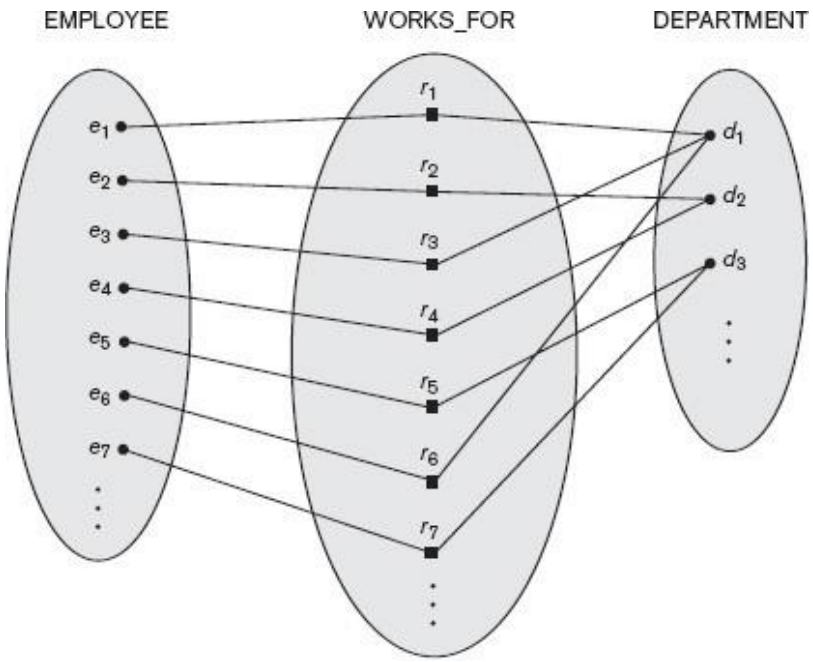
Relationship Types, Sets, and Instances

- A **relationship type** *R* among *n* entity types E_1, E_2, ..., E_n defines a set of associations (a relationship set) among entities from these entity types.
- As with entity sets, a relationship type and its corresponding **relationship set** are referred to by the same name, *R*.

- Each relationship instance r_i in *R* is an association of entities, where the association includes exactly *one* entity from each participating entity type.
- Consider a relationship type WORKS_FOR between the two entity types EMPLOYEE and DEPARTMENT, which associates each employee with the department for which the employee works.
- Each relationship instance in the relationship set WORKS_FOR associates *one* EMPLOYEE entity with *one* DEPARTMENT entity.

WORKS_FOR Relationship

Each relationship instance r_i is connected to the EMPLOYEE and DEPARTMENT entities that participate in r_i.



Employees e_1, e_3, and e_6 work for department d_1; employees e_2 and e_4 work for department d_2; and employees e_5 and e_7 work for department d_3.

Relationship Types in the ER Model

- Relationship types are displayed as *diamond-shaped* boxes, which are connected by straight lines to the rectangular boxes representing the entity types.
- The relationship name is displayed in the diamond-shaped box.

Degree of a Relationship Type

- The **degree** of a relationship type is the *number* of participating entity types.
- Thus, the WORKS_FOR relationship is of degree two.
- A relationship type of degree two is called *binary*, and one of degree three is called *ternary*.
- An example of a ternary relationship is SUPPLY, shown in Figure 8, where each relationship instance r_i associates three entities (supplier *s*, part *p*, and project *j*) whenever *s* supplies part *p* to project *j*.
- Relationships can generally be of any degree, but the ones most common are binary relationships.

SUPPLY Ternary Relationship

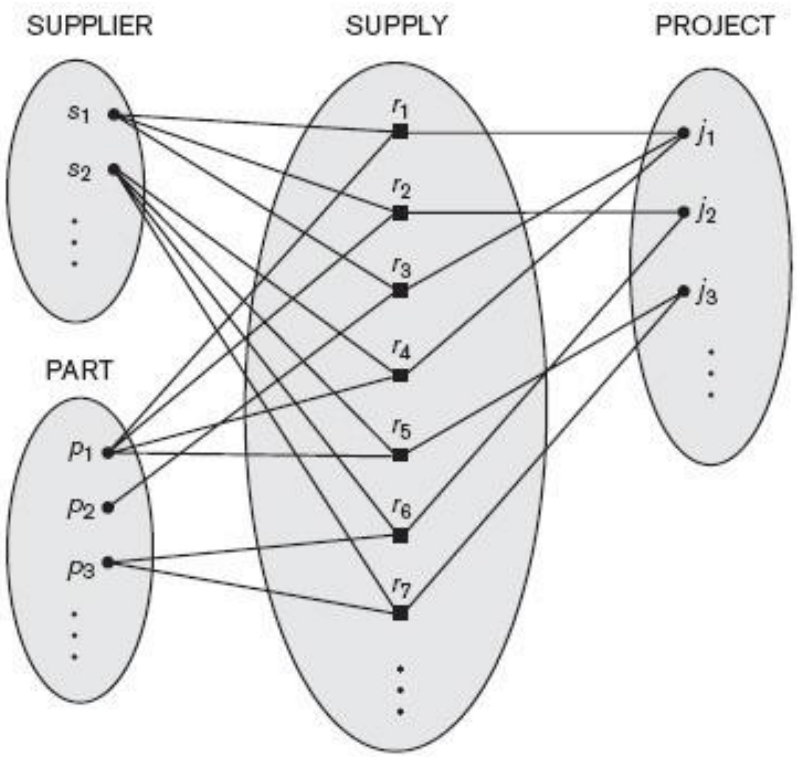


Figure 8: Instances in the SUPPLY Ternary Relationship

All instances r_i connect a singel instance from SUPPLIER, PROJECT and PART.

SUPPLY Ternary Relationship

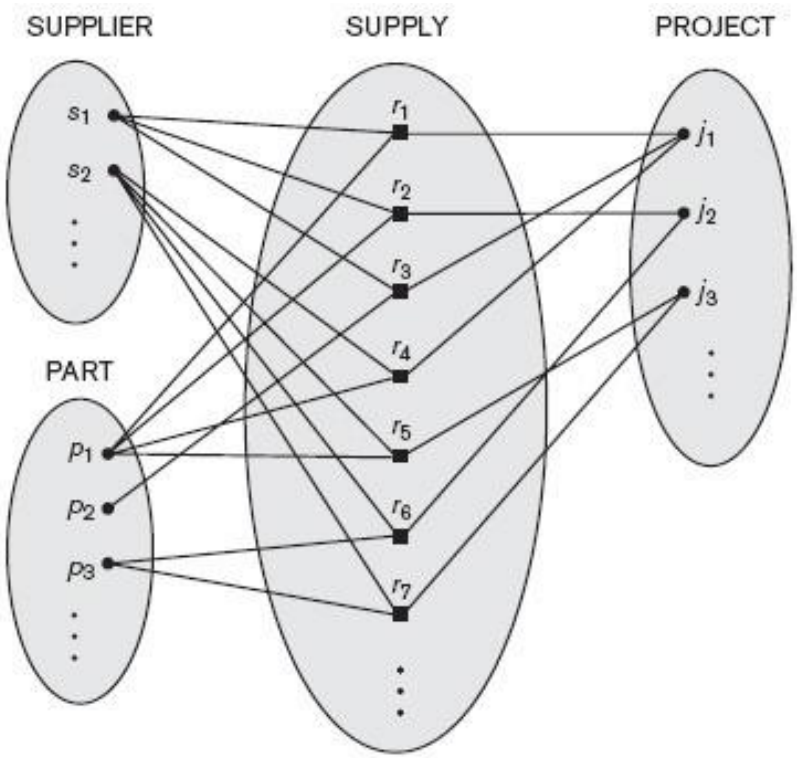


Figure 9: Instances in the SUPPLY Ternary Relationship

Supply r_3 is for part p_2, used in project j_1, supplied by supplier s_1.

Relationships as Attributes

- Consider a **binary relationship** type in terms of *attributes*.
- For WORKS_FOR, there is an attribute called `Department` of the EMPLOYEE entity, where the value of `Department` for each EMPLOYEE is the DEPARTMENT entity where that employee works.
- Thus, the value set for this `Department` attribute is the *set of all DEPARTMENT* entities.

2nd Option

- The alternative is a multivalued attribute `Employee` of DEPARTMENT whose values for each DEPARTMENT entity is the set of EMPLOYEE entities who work for that department.
- The value set of this Employee attribute is the power set of the EMPLOYEE entity set (the subset of employees for *every* department).
- Either of these two attributes (`Department` of EMPLOYEE or `Employee` of DEPARTMENT) can represent the WORKS_FOR relationship type.

Constraints on Binary Relationship Types

- Relationship types have constraints that limit the possible combinations of entities that may participate in the relationship.
- If each employee can work for exactly one department, then describe this (1-to-1) constraint in the schema.
- There are 2 main types of binary relationship constraints: *cardinality ratio* and *participation*.

Cardinality Ratios for Binary Relationships

- The **cardinality ratio** for a binary relationship specifies the *maximum number* of relationship instances that an entity can participate in.
- For relationship type DEPARTMENT: EMPLOYEE is of cardinality ratio *1:N*, meaning that each department employs *any number of* employees, and any employee can work for *only one* department.
- Here, *N* indicates there is *no maximum* number.
- On the other hand, an employee can be related to a maximum of 1 department.

Different Cardinalities

- The possible cardinality ratios for binary relationship types are *1:1, 1:N, N:1*, and *M:N*.
- Figure 10 represents the constraints that an employee can manage one department only and a department can have one manager only.
- The relationship type WORKS_ON (Figure 11) is of cardinality ratio *M:N*, because the rule is that an employee can work on several projects and a project can have several employees.

MANAGES Binary Relationship

MANAGES (1:1) relates a department entity to the employee who manages that department.

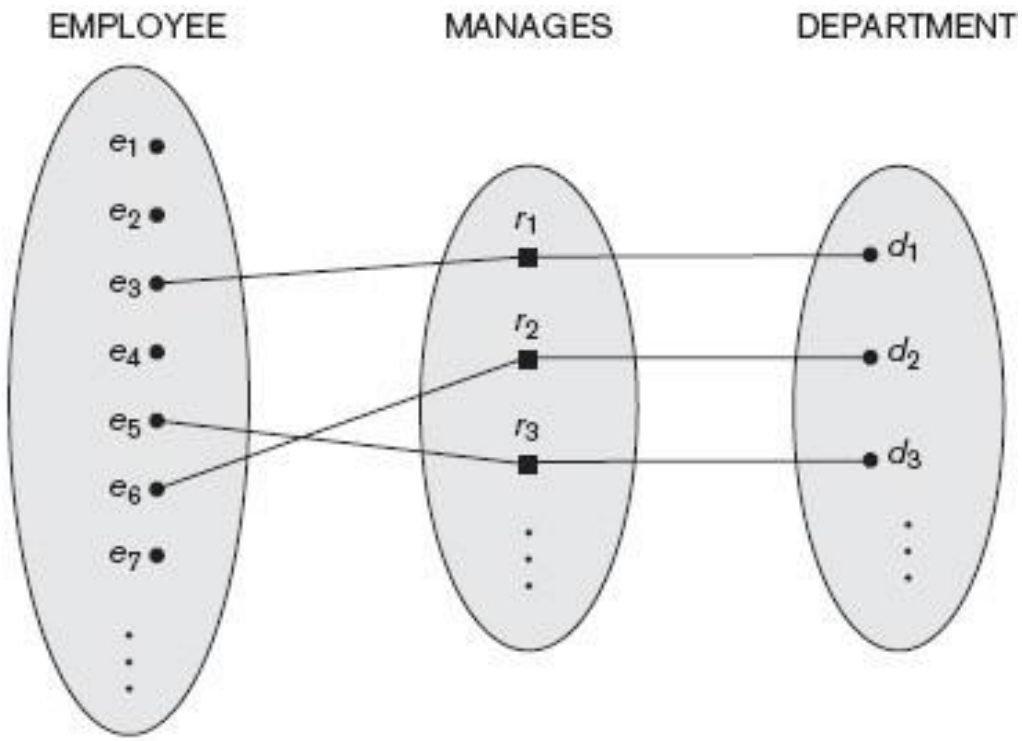


Figure 10: MANAGES Relationship

WORKSON Relationship

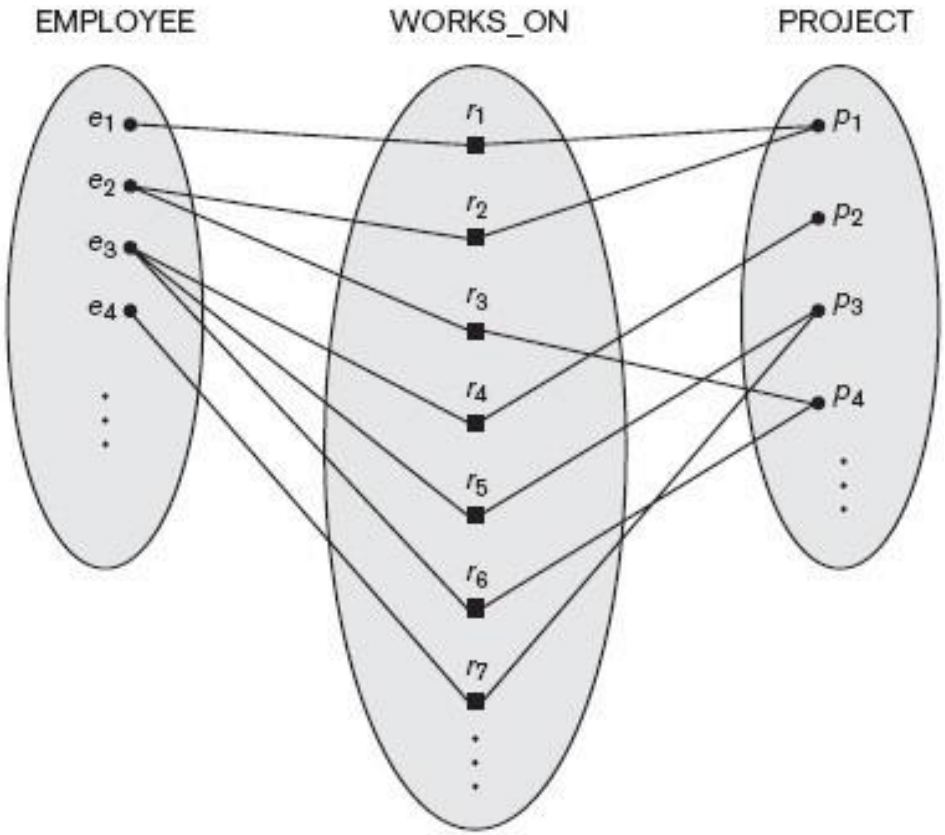


Figure 11: An example of a M:N binary relationship is WORKSON

Employee e3 works on 3 projects using relationships r4,r5,r6 to connect to projects p2,p3,p4.

Cardinalities

- Cardinality ratios for binary relationships are represented on ER diagrams by displaying 1, *M*, and *N* on the diamonds as shown in Figure 1.
- Notice that in this notation, we can either specify no maximum (*N*) or a maximum of one (1) on participation.

Participation Constraints and Existence Dependencies

- The *participation constraint* specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
- This constraint specifies the *minimum* number of relationship instances that each entity can participate in, called the **minimum cardinality constraint**.
- There are two types of participation constraints: *total* and *partial*.

Total Participation

- If every employee *must* work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance.
- Thus, the participation of EMPLOYEE in WORKS_FOR is called **total participation**, meaning that *every* employee entity must be related to a department entity through WORKS_FOR.

Total Participation: Existence Dependency

- Total participation is also called **existence dependency**.
- In Figure 10, we do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is **partial**.
- Thus, *part of the set* of employee entities are related to some department entity via MANAGES, but not all.
- We refer to cardinality ratio and participation constraints as **structural constraints** of a relationship type.

ER Representation

- In ER diagrams, total participation (or existence dependency) is displayed as a double line connecting the participating entity type to the relationship, whereas partial participation is represented by a single line (see Figure 1).
- Notice that in this notation, we can either specify no minimum (partial participation) or a minimum of one (total participation).

Attributes of Relationship Types

- Relationship types can also have attributes, similar to those of entity types.
- To record the number of hours per week for a particular project, include an attribute `Hours` for the WORKS_ON relationship type.
- Include the date on which a manager started managing a department via an attribute `Start_date` for the MANAGES relationship type.

1:N Relationship Attributes

- For a *1:N* relationship type, a relationship attribute can be migrated only to the entity type on the *N*-side of the relationship.
- If the WORKS_FOR relationship has an attribute `Start_date` indicating an employee's start, this is an attribute of EMPLOYEE.
- This is because each employee works for *one* department and thus, participates in at most one relationship instance in WORKS_FOR.

Weak Entity Types

- Entity types that do not have key attributes of their own are called **weak entity types**.
- In contrast, regular entity types that do have a key attribute are called **strong entity types**.
- Entities belonging to a weak entity type are identified by being related to specific entities from another entity type in combination with one of their attribute values.
- We call this other entity type the *identifying* or *owner* entity type, and we call the relationship type that relates a weak entity type to its owner the *identifying relationship of the weak entity type*.

- A weak entity type always has a *total participation constraint* (existence dependency) with respect to its identifying relationship because a weak entity cannot be identified without an owner entity.
- However, not every existence dependency results in a weak entity type.
- For example, a DRIVER_LICENSE entity cannot exist unless it is related to a PERSON entity, even though it has its own key (`License_number`) and thus, is not a weak entity.

- DEPENDENT is used to keep track of the dependents of each employee via a *1:N* relationship with EMPLOYEE.
- In our example, the attributes of DEPENDENT are Name (the first name of the dependent), Birth_date, Sex, and Relationship (to the employee).
- Two dependents of two distinct employees may, by chance, have the same values for Name, Birth_date, Sex, and Relationship, but they are still distinct entities.
- They are identified as distinct entities only after determining the particular employee entity to which each dependent is related.
- Each employee entity is said to *own* the dependent entities that are related to it.

Partial Key

- A weak entity type normally has a *partial* key, an attribute to uniquely identify weak entities related to the same owner entity.
- If we assume that no two dependents of the same employee ever have the same first name, the attribute Name of DEPENDENT is the partial key.
- In the worst case, a composite attribute of all the weak entity's attributes will be the partial key.
- In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their boxes and diamonds with double lines (see Figure 1).
- The partial key attribute is underlined with a dashed or dotted line.

Refining the ER Design for the COMPANY Database

- Refine the database design by changing the attributes that represent relationships into **relationship types**.
- The **cardinality ratio** and **participation constraint** of each relationship type are determined from user requirements.

MANAGES

- MANAGES, a 1:1 relationship type between EMPLOYEE and DEPARTMENT.
- EMPLOYEE participation is *partial*.
- For DEPARTMENT participation, assume that a department must have a manager, which implies *total participation*.
- The attribute `Start_date` is assigned to this relationship type.

WORKS_FOR and CONTROLS

- WORKS_FOR, a 1:N relationship type between DEPARTMENT and EMPLOYEE. Both participations are *total*.
- CONTROLS is a 1:N relationship type between DEPARTMENT and PROJECT.
- The participation of PROJECT is *total*, while that of DEPARTMENT is determined to be *partial*, as some departments control no projects.

SUPERVISION

- SUPERVISION, a 1:N relationship type between EMPLOYEE (in the supervisor role) and EMPLOYEE (in the supervisee role).
- Both participations are determined to be *partial*, after the users indicate that not every employee is a supervisor and not every employee has a supervisor.

WORKS_ON

- WORKS_ON is an M:N relationship type with attribute `Hours`, as projects can have several employees.
- Both participations are determined to be *total*.
- DEPENDENTS_OF is a 1:N relationship type between EMPLOYEE and DEPENDENT, also the identifying relationship for the weak entity type DEPENDENT.
- The participation of EMPLOYEE is *partial*, while that of DEPENDENT is *total*.

Final ER Diagram